

# CLOSUREX: Compiler Support for Correct Persistent Fuzzing

Rishi Ranjan  
rishiranj@vt.edu  
Virginia Tech  
Blacksburg, Virginia, USA

Ian Paterson  
ianp94@vt.edu  
Virginia Tech  
Blacksburg, Virginia, USA

Matthew Hicks  
mdhicks2@vt.edu  
Virginia Tech  
Blacksburg, Virginia, USA

## Abstract

Fuzzing is a widely adopted and pragmatic methodology for bug hunting as a means of software hardening. Research reveals that increasing fuzzing throughput directly increases bug discovery rate. The highest performance fuzzing strategy is persistent fuzzing, which reuses a single process for all test cases by looping back to the start upon completion, instead of exiting. This eliminates all process creation, initialization, and tear-down costs—which are on-par with execution cost. Unfortunately, persistent fuzzing leads to semantically inconsistent program states because process state changes from one test case remain for subsequent test cases. This semantic inconsistency results in missed crashes, false crashes, and overall incorrectness that undermines fuzzer effectiveness.

We observe that existing fuzzing execution mechanisms exist on a continuum, based on the amount of state that gets discarded and restored between test cases. We present CLOSUREX, a fuzzing execution mechanism that sits at a new spot on this state restoration continuum, where only test-case-execution-specific state is reset. This fine-grain state restoration provides near-persistent performance with the correctness of heavyweight state restoration. We construct CLOSUREX as a set of LLVM passes that integrate with AFL++. Our evaluation on ten popular open-source fuzzing targets show that CLOSUREX maintains semantic correctness, while increasing test case execution rate by over 3.5x, on average, compared to AFL++. CLOSUREX also finds bugs more consistently and 1.9x faster than AFL++, with CLOSUREX discovering 15 0-day bugs (4 CVEs).

**CCS Concepts:** • Security and privacy;

**Keywords:** Fuzzing, System Security, Security and Privacy, Software Testing

## ACM Reference Format:

Rishi Ranjan, Ian Paterson, and Matthew Hicks. 2025. CLOSUREX: Compiler Support for Correct Persistent Fuzzing. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, Salt Lake City, USA, 13 pages. <https://doi.org/10.1145/3669940.3707281>



This work is licensed under a Creative Commons Attribution-ShareAlike International 4.0 License.

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0698-1/25/03

<https://doi.org/10.1145/3669940.3707281>

'25), March 30–April 3, 2025, Rotterdam, Netherlands. ACM, Salt Lake City, USA, 13 pages. <https://doi.org/10.1145/3669940.3707281>

## 1 Introduction

Modern applications grow in complexity at an unyielding rate. As code bases continue growing, so too does the technical overhead required to test and validate them [22]. With this growing test debt, industry turns to fuzzing to vet software as an automated supplement to traditional verification [10, 17, 26].

Fuzzing is the process of generating test cases based on a set of seed test cases and running the test case against a program to find bugs. Even short of bug finding, a key aspect of fuzzing is the discovery of “interesting” test cases that explore the nether regions of program functionality. The most popular definition of “interesting” is whether a test case exercises some as-of-yet unseen portion of the program; this is called coverage-guided fuzzing. Incorporating coverage into testing enables orders-of-magnitude improvement in testing effectiveness compared to randomized testing.

Previous work shows that the surest way to increase code coverage via fuzzing is to execute more test cases [1, 11, 18]. There are two ways to increase test case execution rate: decrease program execution time and decrease the time spent executing support code. Recent work has nearly eliminated fuzzing-related overhead incurred during the execution of program code by baking-in coverage tracking [6] and deferring coverage tracking to the uncommon case of an interesting test case [11, 18, 20, 29]. While these systems eliminate most of the wasted cycles spent executing monitoring code, they do not address cycles spent preparing for and cleaning-up from the target’s execution, i.e., execution support.

Minimizing execution support overhead centers on identifying and eliminating all sources of test-case-invariant code execution; this paper targets minimizing process management overhead. At one extreme, the fuzzer creates an entirely new process for each test case. This is a very coarse grain that must create, load, and destroy a process for every test case. This achieves correctness at the expense of performance. At the opposite extreme is persistent fuzzing that reuses a single process for all test cases by jumping back to the program’s start instead of exiting. Ignoring any changes to software-level state across test cases sacrifices correctness for increased performance. In the middle of the spectrum are

approaches that keep parts of the process around between test cases and rely on more fine grain state clean-up. We observe that even the most fine-grain approach is relatively coarse grain.

Our insight is that it is possible to reuse a single process across test cases as long as we preserve the abstraction of each test case getting its own process. Preserving this abstraction requires a form of rollback recovery [8] where, at the completion of a test case’s execution, we rollback to the starting state for the next test case. We design and implement a fuzzer CLOSUREX. CLOSUREX logs changes to program state during execution (e.g., which global variables were modified) and restores them before executing the next test case. By doing this fine-grain, execution-specific state tracking and restoration, CLOSUREX eliminates process creation, loading, and tear-down overheads—while preserving correctness.

We evaluate CLOSUREX using a set of popular open-source programs and compare CLOSUREX against the most popular fuzzer, AFL++. Our evaluation shows that CLOSUREX increases test case execution rate by over 3.5x and discovers bugs 1.9x faster—while maintaining semantic correctness. Across our evaluation, CLOSUREX finds 15 unique bugs across 4 programs, resulting in 4 CVEs, and 7 patches.

CLOSUREX embodies several technical contributions:

- We show that there is a fine-grain, execution specific way to track and undo state changes to create a correct version of persistent fuzzing.
- We implement a set of LLVM passes to reset the state of a program to its starting state after execution.
- We show that AFL++’s persistent mode leads to semantically impossible states, but CLOSUREX maintains semantic correctness.
- We integrate CLOSUREX with the leading fuzzer AFL++ and use that implementation to find 15 0-day bugs.
- We open source CLOSUREX for use and extension by the broader software testing community [23].

## 2 Background

Source-available fuzzing offers the promise of the most efficient and effective fuzzing due to the ability to directly analyze, modify, and integrate with application source code. The two most prevalent use cases of source-available fuzzing are software developers looking to vet their own code and the vetting of open-source code. Previous work leverages this level of program access to provide low-overhead code coverage tracing [6] and to improve effectiveness via sanitizers [6, 24, 30]. **CLOSUREX supplements those advancements with increased performance by eliminating the overhead of process management.**

### 2.1 Compiler Optimizations

LLVM is a language- and platform-agnostic compiler architecture, which allows users to implement and integrate their

own program analysis and transformation passes at compile-time [13]. LLVM works by transforming the source code into an intermediate representation called LLVM IR. The LLVM IR is a strongly-typed assembly-like intermediate representation with an added catch of infinite virtual registers. The language and platform agnostic optimizations are applied directly on the LLVM IR. The way that virtual registers are utilized is a feature of Static Single-Assignment (SSA), which is a methodology for structuring code during parsing to ensure every instance of a register is assigned-to only once in IR. Developers implement tooling in the form of LLVM optimizations (opt) passes written in C++, which process and modify the LLVM IR.

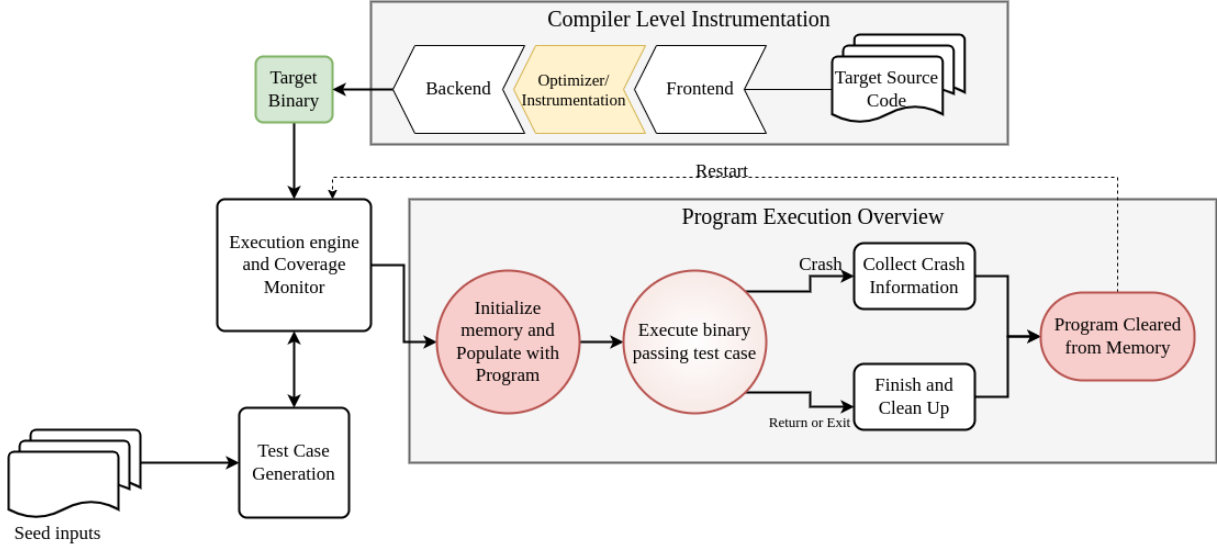
LLVM based code modification is now the state of the art methodology for implementing instrumentation and tooling for fuzzers [6, 9, 14, 24, 25, 30]. LibFuzzer and AFL++ both implement LLVM-based instrumentation in the form of pcguard instrumentation [6, 14, 35]. Pcgard is used to measure the state of the program at certain points in execution, and is used for calculating the code coverage during execution to feed this information for analysis and test generation. Other forms of instrumentation are in the form of error detectors [24, 30]. **CLOSUREX is a set of LLVM passes that integrate with the fuzzer AFL++.**

### 2.2 Fuzzing Overview

Fuzzing is the process of feeding under-constrained test cases into an application and monitoring the resulting execution. The execution results provide insight into the behavior of the application with the end goal of finding crashes and triaging the crash-inducing test case to uncover the associated bug. The process of fuzzing consists of four steps [35]:

1. Program *instrumentation*
2. Test case *generation*
3. Program *execution* and monitoring
4. Feedback analysis and *triage*

In practice, applications are fuzzed in either binary-only or source-code-available methods. These two different modes mean that we have two different ways to harness the target program and measure its execution. Binary-only fuzzing requires analysis of the binary program to disambiguate structure and inject coverage tooling at the basic block level utilizing binary rewriting [19, 33]. This is done traditionally in the form of callbacks [9] and system signals [18, 20] at the entry to each basic block to determine code coverage. Binary-only fuzzing also requires either dynamic [36] or static [4] rewriting to harness the target and insert coverage tracing code. Alternatively, source-available fuzzing harnesses the target program and inserts coverage tracing logic directly into the source code [6, 9, 13, 25, 28, 30]. This allows source-available fuzzing to achieve both higher performance and higher effectiveness in bug finding compared to binary-only fuzzing, making it the most common use case within industry. Microsoft, Google, and the U.S. Government all require



**Figure 1.** Traditional fuzzing loop: red portions represent the current performance bottlenecks for source-available fuzzing targeted by CLOSUREX.

| Management Mechanism      | OS Support |         |       | Correct | Target   | Performance |
|---------------------------|------------|---------|-------|---------|----------|-------------|
|                           | Linux      | Windows | MacOS |         |          |             |
| Fresh Process - system()  | ✓          | ✓       | ✓     | ✓       | App      | 1.00        |
| Fresh Process - fork/exec | ✓          | ✗       | ✓     | ✓       | App      | 1.27        |
| Forkserver                | ✓          | ✗       | ✓     | ✓       | App      | 3.27        |
| Kernel-based              | ✓*         | ✗       | ✗     | ✓       | App      | 6.22        |
| Persistent                | ✓          | ✓       | ✓     | ✗       | Lib      | 14.22       |
| CLOSUREX                  | ✓          | ✓       | ✓     | ✓       | App, Lib | 13.20       |

**Table 1.** Comparison of available process management techniques. CLOSUREX is the only approach that works across OSES, maintains correctness, can use both whole applications and functions as targets, and is performant. \*Table 2.

fuzzing within their SDLC for validating the security of their tools and applications [10, 17, 26].

Test case generation is the process of taking an initial seed corpus and using it to generate similar test cases that differ just enough to explore new parts of a program. Ideally, test case generation will change portions of the seed test case that govern conditional statements, exposing unseen program behavior. The process of generating new test cases based on the current coverage is coverage-guided fuzzing. Test case generation falls into two styles: generational [3, 31] and mutational [2, 6, 9, 14, 26, 35]. Generational fuzzers build test cases from scratch, often paired with an abstract model of acceptable test case as a part of grammar-based fuzzing [27, 32], while mutational fuzzers apply random mutations to saved test cases to build new ones.

Triage consists of processing the feedback obtained from executing the target with a test-case and determining whether the test-case is “interesting” and should influence future mutations. Crash triage is the process of isolating a crash found

by the fuzzer, and determining its root-cause. **CLOSUREX is a coverage-guided, mutational, source-available fuzzer.**

### 3 Motivation

Faster execution of the target program is one of the most important criteria for effective fuzzing. In order to discover bugs, a fuzzer needs to execute a large number of test cases. Previous work shows that fuzzers spend over 90% of their time on test case execution [18]. Thus, improving the execution speed of the target directly improves the fuzzer’s effectiveness. **The great success on reducing the overhead of program instrumentation to speed up fuzzing [6, 11, 18–20] has made the overhead due to process management the main source of run-time overhead in fuzzing.** In this section, we analyze the various process management techniques used in fuzzers in terms of performance and correctness, as Table 1 highlights.

|              | Kernel-based snapshot | AFL++ |
|--------------|-----------------------|-------|
| Linux 4.8.10 | ✓                     | ✓     |
| Linux 5.15.0 | ✗                     | ✓     |
| Linux 6.2.0  | ✗                     | ✓     |
| Windows      | ✗                     | ✓     |
| MacOS        | ✗                     | ✓     |

**Table 2.** Comparison of kernel-based snapshot and AFL++ portability. CLOSUREX extends AFL++ in a kernel agnostic way.

### 3.1 Fresh Process

The most primitive execution mechanism, which is the go-to technique of target execution in Windows-based fuzzers, is per-test-case process creation. In this technique, a new process, its related data structures in kernel, and all its memory pages are allocated and populated with the binary for every test case. Creating a new process for each test case provides a consistent starting state for all test cases, but incurs a huge overhead due to time spent creating, initializing, and tearing down the process. The Linux analog to fresh process creation on Windows is the fork and exec system calls, which copy the parent process (i.e., the fuzzer process) and load the target program’s binary in the resulting child process. We observe that this approach erases and reloads far more of the target program than is necessary, wasting fuzzer time.

### 3.2 Forkserver-based

To deal with the high overhead of target initialization, AFL-based Linux fuzzers employ a forkserver, a significant improvement over simple fork/exec-based execution. A forkserver initially forks and loads the target binary into memory, pausing at the beginning of its main function. The forkserver then waits for a new test case from the fuzzer. When it is received, the forkserver forks itself (i.e., creates a copy of the target’s process at that point in execution) and continues executing of the child into the target’s original main, using the received test case. The forkserver is more fine-grain than creating a fresh process as it ensures that the process initialization steps are done—only once—and provides a transparent page-level rollback of test-case-induced state changes via copy-on-write. Even though a forkserver does away with process loading costs, it still suffers from page-level duplication/management costs, process tear-down costs, as well as the kernel-level cost of process duplication. We observe that this technique is still relatively coarse-grain as it works at the page level and also relies on creating many processes.

### 3.3 Kernel-based

An operating-system-specific improvement on the forkserver is kernel-based snapshotting [34]. Kernel-based snapshotting makes fuzzing process management a first class abstraction with a fuzzing specific fork system call. The fuzzing

fork leverages knowledge of the fuzzing use case to reduce the amount of process state that needs to be protected and tracked across test cases. This provides a modest performance improvement, but only works for open source operating systems and frequent updates, as the compatibility results in Table 2 show. Real world adoption of fuzzing advancements shows that any fuzzing execution approach, while being fast and effective, must also be portable and maintainable across platforms. Apart from compatibility, we also observe that kernel-based snapshotting is not fine-grain enough, that there is no need for multiple target processes, and, most of all, there is no need for specialized OS support.

### 3.4 Persistent (i.e., Single Process)

AFL++ provides a test case execution mode through LLVM instrumentation called persistent mode. Persistent fuzzing, unlike traditional fork-based fuzzing, reuses a single process for all test cases.<sup>1</sup> Using only a single process for the entire fuzzing campaign improves test-case execution rate at the cost of breaking inter-test-case isolation, i.e., the effects on program and process state from one test case carry over to all subsequent test cases. While persistent fuzzing eliminates process management overhead [5, 6, 12], breaking this isolation leads to program states inconsistent with program semantics. Semantically inconsistent states occur when a test cases starts in a program state not possible given the program’s code due to the latent effects of a previous test case’s execution. The impact of semantically inconsistent execution comes in the underlying idea of *stale state* (shown in Figure 2). Semantically inconsistent executions result in:

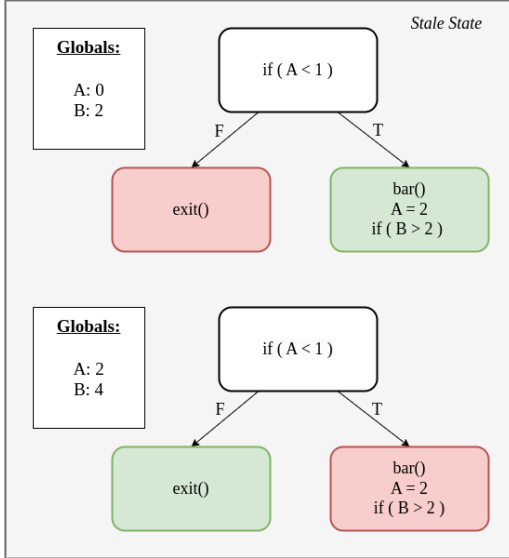
- **Missed crashes:** leave programs vulnerable as testers require crashes to identify bugs.
- **False crashes:** waste tester time by requiring them perform in-depth analysis of test cases and program behavior that does not lead to a bug.
- **Non-reproducibility:** reduce trust in fuzzing as a useful tool for testing as regression testing tool as program behavior is now influenced by test case order.

To prevent semantically inconsistent executions and their knock-on effects, developers must leverage a deep understanding of the code to manually track and revert all such persistent state changes, which is challenging even at the library level [14, 25]. Unfortunately, such expertise and accuracy does not scale to applications.

## 4 CLOSUREX Design

We observe that there is a gap between the *no-state-restoration* of persistent mode and the *general-purpose-state-restoration* of process-level approaches. We posit that a best of both

<sup>1</sup>In practice, developers limit the number of test cases to use a single process for before creating a fresh process as a hack to deal with cross-test-case state contamination. This number can be as low as one to maintain correctness.



**Figure 2.** Example of stale state scenario, in this situation, after the first iteration we get locked out of the true branch which could hold deeper points of code to explore with a certain input and without accounting for this we end up wasting fuzzing efforts.

worlds approach is possible through combining static and dynamic analysis to make self-contained, naturally restartable programs. We advocate baking into a program the ability to track its own state changes, at a fine grain, and reset them between test cases, enabling automatic, correct, whole-application persistent fuzzing—without reliance on special OS support. Doing so achieves high performance while retaining semantic correctness.

To validate this best of both worlds solution, we design CLOSUREX. CLOSUREX is a compile-time instrumentation tool which transforms unmodified target programs into naturally-restartable, fuzzing-oriented programs. CLOSUREX tackles two high-level challenges:

- Injecting a persistent fuzzing loop into the program so that a single process can support executing multiple test cases.
- Adding fine-grain program state tracking and rollback between test cases to ensure test case isolation.

#### 4.1 Injecting a Persistent Fuzzing Loop

All executable C programs have a primary function named `main` that the C runtime calls after initialization to start execution. After execution, the `main` function returns and the operating system reaps the process's resources. Instead of returning to the C runtime, we wrap CLOSUREX's `main` around the target's `main` such that the target returns to CLOSUREX.

A second exit path is also possible in the case of abnormal program termination. Programs that detect malformed input often call `exit` to quickly pass control to the OS, which then reaps the target process. Making matters more challenging

is that `exit`-based termination is common in fuzzing since a majority of the test cases are malformed due to randomized mutation and the under-constrained nature of fuzzing.

CLOSUREX handles `exit` calls by the target in a method similar to a try catch block in modern programming languages. Before calling the target's `main`, CLOSUREX sets a restore point using the C language feature `setjmp`. `setjmp` effectively saves the current state of the stack and registers in case the ensuing execution needs to be unwound. Then, CLOSUREX replaces the target's `exit` calls with `longjmp` calls, which restore the stack to the values saved by the `setjmp`. CLOSUREX then rollbacks all other state changes made by the target. Note that `exit` calls performed by libraries are considered critical which need handling by the fuzzer, so CLOSUREX leaves those intact.

The last part of injecting a harness is communicating with the fuzzer. CLOSUREX inserts code that coordinates with the fuzzer to wait for a new test case to be generated. Beyond creating a synchronized channel to pass test cases to the target, this informs the fuzzer that new coverage data is available from the previous test case.

#### 4.2 State Restoration For Correct Execution

Whenever a program executes, it modifies program state during its execution. Without restoring software state, semantically impossible states occur that hide crashes and create false crashes. To ensure semantically correct starting state for every test case, we identify the major sources of program state which we address and restore at the end of every execution.<sup>2</sup>

**Program stack.** The stack is the address space in the process where the local variables, as well as subroutines' information are stored during program run time. This is a static memory section of the process and a stack pointer is used to increase or decrease the size of the stack during execution. When a function is called, the caller stores the return address and the base pointer of its own stack frame on the stack. The callee function then allocates space on the stack by reducing the stack pointer appropriately. The saved return address and caller's stack pointer value are used during function return to restore the stack back to its caller. Thus, the stack holds important program state during run time. Incorrect restoration of the stack after every iteration leads to the program running out of space or collisions with the heap.

To deal with this, there are two cases that CLOSUREX must handle: normal return flows and `exit` flows. In normal flows, execution returns back to CLOSUREX's persistent loop via a series of return statements. This naturally clears the stack.

<sup>2</sup>CLOSUREX is an extensible platform for correct persistent fuzzing. The types of state addressed in this section are what we saw with a diverse set of real-world programs. Developers can use these state tracking and restoration patterns to handle other scenarios not accounted for here.

In an exit flow, CLOSUREX's added `longjmp` uses the values saved by `setjmp` to restore the stack.

**Program heap.** The heap is a dynamic memory section which is used to fulfill the memory requests by the program at run time. These requests are made using calls to `malloc`, `calloc`, `realloc`, etc. The memory obtained by these calls must later be returned to the C runtime using `free`. However, most of the end user programs that we explore have memory leaks—because they rely on the operating system to reap any outstanding dynamic memory after execution completes. Thus, these memory leaks are not harmful in such isolated executions due to the kernel taking care of it, but devastating for persistent execution as the memory leaks add up and consume all memory. This leads to false crashes.

We identify that memory allocations done during a process's lifetime originate from different sources. The C standard library `libc` as well as other dynamically-loaded libraries also allocate dynamic memory. If such memory is freed after every iteration, it can lead to undefined behaviour in the target. Thus, CLOSUREX hooks all dynamic memory related functions in the target and frees and allocates, but unfreed memory by the target between test cases.

**Global memory.** Global memory is used by programs to store data that is accessible to all functions or to store function-scoped variables that need to persist across function calls. This is a static memory region as it is present in the executable itself. Global memory, thus is an important source of program state—and one most harmed by persistent fuzzing. After the execution, the kernel cleans up all the process-related memory pages which also includes global memory pages. During fresh process execution, the target always starts with a non-contaminated global memory state as it is loaded directly from the executable. However persistent fuzzing uses the same process for a large number of target executions and contaminated global variables from previous execution lead to both missed and false crashes.

CLOSUREX takes a fine-grain approach to global tracking and restoration to increase performance. CLOSUREX narrows down the restoration to only the relevant sections of the global memory that are possible to be changed during execution. CLOSUREX creates a snapshot of potentially modifiable global memory region of the target and restores it after every target iteration using byte-level, rather than page-level writes, improving performance.

**File descriptors.** In Linux, every open file is associated with a file descriptor, which is handled by the kernel on behalf of the process. The kernel sets an upper limit on the number of file descriptors a process can use. When a process exits, the kernel closes all the open file descriptors, thus cleaning up all zombie file handles for the process. Due to the limit on active file descriptors, in persistent fuzzing, even a single file descriptor left open after an iteration leads to the process running out of file descriptors, creating false crashes.

| CLOSUREX Pass  | Functionality                                                 |
|----------------|---------------------------------------------------------------|
| RenameMainPass | Rename target's main                                          |
| HeapPass       | Inject tracking of target's heap memory                       |
| FilePass       | Inject tracking of target's file descriptors                  |
| GlobalPass     | Move target's writable globals into a separate memory section |
| ExitPass       | Rename target's exit calls                                    |

Table 3. CLOSUREX LLVM Passes

We observe that during fuzzing, the file handles opened by the target fit into two sets. First is the handles which are opened once during the initialization of the target i.e., they're part of the initialization process. Closing and reopening such handles after every iteration leads to inefficiency and a better approach is to reset the handle to its starting position. The second set of file handles are the ones opened by the target during the fuzzing relevant execution. For correct execution, CLOSUREX tracks such file handles during the target's execution and closes them between test cases.

## 5 CLOSUREX Implementation

We implement CLOSUREX in five compile-time passes for LLVM, as shown in Table 3. We choose LLVM due to its modularity and ease of development, but our approach can be extended to any compiler toolchain like GCC. Our instrumentation is compatible with any coverage-tracking instrumentation, and to benchmark the fuzzing performance, we compile the target with AFL's Sanitizer Coverage Guard on top of CLOSUREX's instrumentation.

### 5.1 CLOSUREX Harness

To implement correct persistent fuzzing, we inject a harness to loop over the target code for each test case. Our harness initializes the target and stores relevant program state before target execution. After every iteration of the target, our harness restores the state of the target program. Listing 1 depicts the harnessed target.

```

1 target_main(argc, argv) {
2     // Input parsing related code
3     // Body of target program
4 }
5 // Buffer for saved context
6 jmp_buf longjump_buf;
7
8 main(argc, argv) {
9     global copy =
10         copy_writable_global_sections();
11     while(1) {
12         // Wait fuzzer to collect coverage and
13         // new test case
14         wait_for_fuzzer();
15         if (setjmp(longjump_buf)) {

```

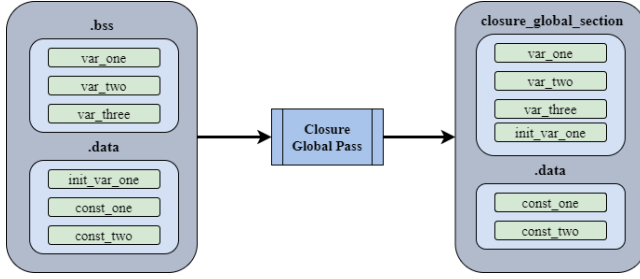


Figure 3. The transformation performed by CLOSUREX's Global pass

```

14     ;// Target called exit()
15 } else {
16     // This is the main function of target
17     target_main(argc, argv);
18 }
19 // Stack already restored; restore
    everything else
20 restore_global_sections();
21 reset_heap_memory();
22 close_open_file_handles();
23 }
24 }

```

Listing 1. Target injected with the CLOSUREX harness at compile time.

## 5.2 Renaming main

We rename the target's main function to `target_main`. CLOSUREX's stub harness then calls `target_main` from its own main function in a loop. We perform this function renaming using our LLVM Module pass `RenameMainPass`. This pass operates on the module and tries to locate a function called `main` in the module. `RenameMainPass` renames the function using the `setName` method call on the `Function` object. In cases where the target takes fuzzed input from a file, the arguments passed to the target's main function are the same as command-line arguments, i.e., `argv` and `argc`. When the target takes the fuzzed input directly from the command line, `target_main` includes logic to replace the appropriate `argv` with the test case supplied by the fuzzer.

## 5.3 Resetting the Program Stack

CLOSUREX's stub harness implicitly resets the stack of the target. Since CLOSUREX's harness's main function replaces the target's main function, the calls to the target's main function are semantically correct. In the case of normal execution end (return from `target_main`), the series of return statements which eventually flow back to the harness unwind the stack automatically.

## 5.4 Dealing with Abnormal Program Exits

Apart from returning normally after execution of the target's main, a target can use the `exit` system call to exit the process immediately—without unwinding the stack. This is a typical behaviour in the case of failed file parsing.

To deal with this, CLOSUREX uses `setjmp` and `longjmp` from the C language. We set a jump target at the beginning of our harness's main function using the `setjmp` function, which also stores the values present in general purpose registers at the jump target. During compile-time, we replace all the calls to `exit` in the target code with our function wrapper `exitHook`. This is done by `ExitPass` using the `replaceAllUsesWith` method. The `exitHook` function wrapper calls `longjmp` to the `setjmp` target with the exit status. The `longjmp` function transfers the control flow back to our harness's execution loop and also restores the saved values into the general purpose registers, restoring the stack along with them. CLOSUREX hooks `exit` calls only in the instrumented source code of the program, and thus this has no effect on any exception handling code or `exit` calls in any externally loaded libraries like standard library `libc`.

## 5.5 Resetting Global Memory

As shown in Figure 3, to reset the global memory of the target, our compile-time pass `GlobalPass` first identifies the potentially modifiable global variables in the target, and then moves all such variables into a new section in the target, which we name as `closure_global_section`. In order to identify potentially modifiable variables in an LLVM module, we iterate over all the global variables in a module and call `isConstant` method on it which returns true if it is a constant and false otherwise. If the variable is not a constant, we move it to the new section using the `setSection` API call in LLVM. Our approach greatly reduces the size of the global section by excluding entities such as immutable strings from `closure_global_section`.

During the fuzzing campaign, the address of the new global section is passed as the environment variable `CLOSURE_GLOBAL_SECTION_ADDR`, and the size is passed as the environment variable `CLOSURE_GLOBAL_SECTION_SIZE`. The address and size of `closure_global_section` can be obtained using an ELF parsing tool like `readelf`. Observe in Figure 4, before the execution of target's main function, the new section is copied into a buffer by the CLOSUREX's harness. Between test cases, this buffer is used to restore the writable global section, thus rolling back any modifications.

## 5.6 Resetting Dynamic Memory

To manage the dynamic memory of the target, we deal with two sources of leaks. Not only do we free all the memory leaks by the target, we also ensure that no allocated memory already freed by the target is freed by our mechanism. To achieve this, CLOSUREX tracks all the memory allocations

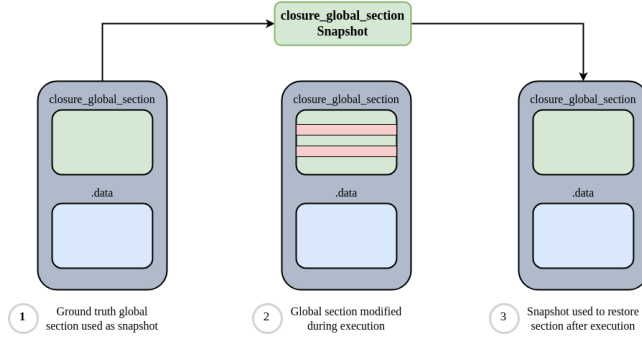


Figure 4. CLOSUREX global resetting procedure

and deallocations by the target by replacing all instances of the following functions in the HeapPass:

- `malloc` → `myMalloc`
- `realloc` → `myRealloc`
- `calloc` → `myCalloc`
- `free` → `myFree`

HeapPass operates on an LLVM module. For every module, we declare a new function for `myMalloc`, `myCalloc`, `myRealloc` and `myFree` in the module using the `getOrInsertFunction` method in the module. These declarations are resolved during the linking-phase of the compilation, as they're defined in CLOSUREX's harness. We then perform the replacements using the `replaceAllUsesWith` method on the Function object for every `malloc`-family function. Figure 5 shows an example. When the target requests memory using `malloc`-family functions, our wrapper stores the pointer returned into a hash map structure. Whenever a pointer is freed by the target, it is removed from the hash map structure by our custom `free` function `myFree`.

After the target's main function is executed and the control flow returns to CLOSUREX's harness, we iterate over all the pointers still stored into our hash map and free them. This prevents the target from leaking dynamic memory.

### 5.7 Closing Open File Handles

CLOSUREX deals with all file descriptors in the target with function replacement techniques similar to resetting dynamic memory of the target. We replace file opening routine `fopen` with `fopen_hook` which tracks and stores every open file handle into a hash map. We replace calls to `fclose` with `fclose_hook` which deletes the file handle from our hash map when it is closed. This is done by our FilePass Function Pass. After every test case, CLOSUREX iterates over the hash map and closes all the open file handles. This pattern described is useful for managing other resource handles which can be acquired on Linux like sockets and shared memory.

## 6 Evaluation

CLOSUREX provides developers the performance of persistent fuzzing with the correctness of fresh process fuzzing. Thus, the evaluation systematically explores those two criteria.

1. **Correctness:** Is test case execution the same no matter if it was executed in a fresh process versus being executed in the same process after many other test cases?
2. **Performance:** What is the improvement that CLOSUREX's instrumentation achieves over the state-of-the-art correct process management approach in terms of test case throughput and bug finding?

### 6.1 Evaluation Setup

We compare CLOSUREX with the process management mechanism of the most popular Linux fuzzer AFL++. CLOSUREX instrumented targets are fuzzed with AFL++ and no modifications are needed in AFL++ to support CLOSUREX's instrumentation. Both CLOSUREX and AFL++ use the same coverage tracing and seed mutation mechanisms. Doing this isolates the impact of CLOSUREX's persistent execution mechanism. **The ideal outcome is CLOSUREX finds the same crashes and coverage as AFL++ (since the underlying fuzzer is the same and CLOSUREX maintains correctness), but much faster than AFL++.** Finding new crashes faster is important as it reduces the resources needed for a given exploration rate or decreases the vulnerability time-window of a program.

To ensure consistency and reproducibility, we perform all experiments on Microsoft Azure cloud instances of type Standard\_DS1\_v2, with 3.5 GB memory and a single CPU core running Ubuntu 20.04 LTS. In order to account for randomness and add statistical rigour to our results, we perform 5, 24-hour trials of each benchmark/fuzzer configuration.

The evaluation consists of ten diverse, open-source benchmark programs, common to fuzzing platform evaluations [16, 20, 26]. The benchmarks selected for the evaluation satisfy the following criteria:

- **Extensive fuzzing:** each of the benchmark has been part of multiple peer-reviewed academic works or well fuzzed open-source fuzzing benchmarks. This enables cross comparison as well as making any bugs we uncover significant as they have not been discovered previously under extensive fuzzing.
- **Diversity:** benchmarks vary in complexity and task.
- **Rich variety of input formats:** Table 4 shows that our benchmarks have a diverse set of input formats.

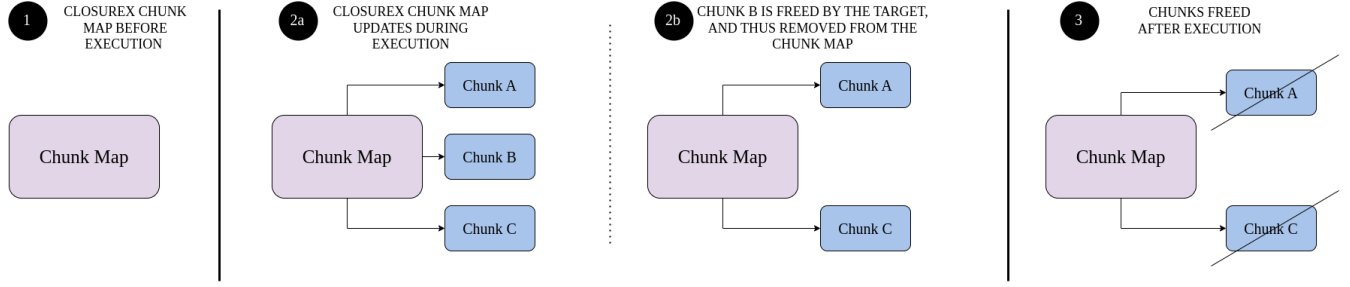


Figure 5. CLOSUREX heap resetting procedure during runtime

| Benchmark   | Input Format | Executable Size |
|-------------|--------------|-----------------|
| bsdtar      | tar          | 4.7 M           |
| libpcap     | pcap         | 2.4 M           |
| gpmf-parser | mp4 (GoPro)  | 720 K           |
| libbpf      | bpf object   | 1.9 M           |
| freetype    | ttf          | 4.6 M           |
| gifttext    | gif          | 232 K           |
| zlib        | zlib archive | 260 K           |
| libdwarf    | ELF          | 2.8 K           |
| c-blosc2    | bframe       | 12 M            |
| md4c        | markdown     | 652 K           |

Table 4. Evaluation benchmarks.

| Benchmark   | CLOSUREX | AFL++ | Speedup | $\rho$ value |
|-------------|----------|-------|---------|--------------|
| bsdtar      | 379M     | 93M   | 4.09    | 0.0079       |
| libpcap     | 565M     | 201M  | 2.81    | 0.0079       |
| gpmf-parser | 456M     | 193M  | 2.36    | 0.0079       |
| libbpf      | 884M     | 212M  | 4.6     | 0.0079       |
| freetype    | 493M     | 168M  | 2.94    | 0.0079       |
| gifttext    | 1118M    | 233M  | 4.79    | 0.0079       |
| zlib        | 1117M    | 293M  | 4.00    | 0.0079       |
| libdwarf    | 913M     | 265M  | 3.44    | 0.0079       |
| c-blosc2    | 905M     | 213M  | 4.24    | 0.0079       |
| md4c        | 538M     | 215M  | 2.50    | 0.0079       |
| Average     |          |       | 3.53    | 0.0079       |

Table 5. Average number of test cases executed in 24 hours along with the statistical significance  $\rho$  value for the Mann-Whitney U-test.  $\rho < 0.05$  indicates statistical significance [15].

## 6.2 Test Case Execution Rate

Previous work shows that the test case throughput is the most significant part of efficient and successful fuzzing campaign [18]. CLOSUREX enables the target to isolate its process initialization, loading, and tear-down procedures from the main fuzzing loop, thus improving test case throughput. In order to measure this performance gain, we compare CLOSUREX with AFL++’s forkserver process management mechanism, which is the fastest correct process management mechanism in fuzzing.

We run 5, 24-hour trials for our benchmarks and report the average number of test cases executed by both fuzzers, as well as the improvement CLOSUREX achieves over AFL++. Table 5 summarizes the throughput results. On an average, CLOSUREX executes more than **3.5x** as many test cases than AFL++.

## 6.3 Code Coverage Improvement

Given that CLOSUREX is equivalent to fresh process execution, just much faster, CLOSUREX and AFL++ will cover the same portions of code (ignoring randomness), but CLOSUREX should do it much faster. Code coverage is an important metric of coverage-guided fuzzing. It guides the fuzzer to test deeper and more complex logic in the target. Our coverage mechanism is same as AFL++, which has its own hitcount-based edge coverage collection implementation loosely based on LLVM’s Sanitizer Coverage Guards.

| Benchmark   | CLOSUREX | AFL++  | % Improvement | $\rho$ value |
|-------------|----------|--------|---------------|--------------|
| bsdtar      | 18.13%   | 13.80% | 31.4          | 0.031        |
| libpcap     | 15.64%   | 15.22% | 2.76          | 0.547        |
| gpmf-parser | 14.43%   | 13.82% | 4.41          | 0.222        |
| libbpf      | 6.45%    | 6.34%  | 1.8           | 0.015        |
| freetype    | 16.30%   | 16.11% | 1.17          | 0.150        |
| gifttext    | 27.57%   | 27.21% | 1.32          | 0.003        |
| zlib        | 36.22%   | 35.91% | 0.87          | 0.111        |
| libdwarf    | 5.20%    | 5.04%  | 3.34          | 0.020        |
| c-blosc2    | 2.12%    | 1.76%  | 20.41         | 0.093        |
| md4c        | 82.17%   | 82.08% | 0.11          | 0.010        |
| Average     |          |        | 7.8           | 0.079        |

Table 6. Edge Coverage percentage of CLOSUREX and AFL++ for different benchmarks, along with the  $\rho$  value for Mann-Whitney U-test.  $\rho < 0.05$  indicates statistically significant results.

Table 6 summarizes the average code coverage for CLOSUREX and AFL++ across benchmarks. On an average, CLOSUREX covers **7.8%** more edges. Mann-Whitney U-test indicates statistically significant improvement across five of the benchmarks.

## 6.4 Time-to-bug Improvement

Even though faster execution and quicker code coverage demonstrate the efficiency of a fuzzer, its practical value

| Benchmark   | CLOSUREX  | AFL++     | Bug Type                   |
|-------------|-----------|-----------|----------------------------|
| c-blosc2    | 7148 (4)  | 11896 (2) | Null Ptr Deref.            |
| c-blosc2    | 25358 (4) | 12471 (2) | Null Ptr Deref.            |
| c-blosc2    | 7442 (4)  | 18299 (2) | Null Ptr Deref.            |
| c-blosc2    | 22957 (1) | 16097 (1) | Null Ptr Deref.            |
| gpmf-parser | 798 (5)   | 1386 (5)  | Division by Zero           |
| gpmf-parser | 43178 (3) | 2365 (1)  | Unaddressable Access       |
| gpmf-parser | 2368 (5)  | 20340 (5) | Unaddressable Access       |
| gpmf-parser | 33028 (5) | 22721 (5) | Division by Zero           |
| gpmf-parser | 6144 (5)  | 6238 (4)  | Invalid Write              |
| gpmf-parser | 14025 (2) | 5493 (2)  | Invalid Read               |
| libbpf      | 61 (5)    | 91 (5)    | Null Ptr Deref.            |
| libbpf      | 515 (5)   | 552 (5)   | Null Ptr Deref.            |
| libbpf      | 2278 (5)  | 9964 (4)  | Null Ptr Deref.            |
| md4c        | 8489 (5)  | 22254 (4) | Memcpy with negative size  |
| md4c        | 44714 (3) | 54342 (2) | Array out of bounds access |

Table 7. Time (sec) for AFL++ and CLOSUREX to find bugs (number of trials which found that bug) for each fuzzer.

comes from finding bugs—and finding them faster. In order to showcase the practical value of CLOSUREX, we evaluate AFL++ and CLOSUREX, on their speed and effectiveness in finding bugs. As a part of the performance and coverage evaluation, we find 15 0-day bugs in the latest source-code across 4 different benchmarks. Table 7 summarizes the bugs discovered. When CLOSUREX and AFL++ find the same bugs, CLOSUREX does so **1.9x** faster. CLOSUREX also finds bugs more consistently than AFL++, with 25% more trials finding bugs.

**0-days and bug disclosure** All the bugs reported above in Table 7 are 0-day bugs. We reported all the bugs to their respective developers. As of writing, bugs in c-blosc2 and gpmf-parser have been fixed. The developers of libbpf acknowledged the bugs. We have not heard back from the md4c developers. We have been assigned 4 CVEs (CVE-2023-37185, CVE-2023-37186, CVE-2023-37187, CVE-2023-37188).

One of the bugs found during our fuzzing campaign is a null pointer dereferencing bug in libbpf. libbpf is a Linux kernel library used to load and run bpf object files into the Linux kernel in a sandboxed environment. The library tries to parse the relocation section of the crashing ELF object, and reads the data of relocation section, which returns a NULL pointer. When this NULL pointer is accessed in the library, it causes a crash.

## 6.5 CLOSUREX Correctness

To validate that CLOSUREX correctly restores program state between test cases, we show that no matter the number or order of test cases preceding a test-case-of-interest, it is equivalent, semantically, to running that test case in isolation, with a fresh process. We divide this into two kinds of equivalence with fresh process execution:

1. Dataflow equivalence: program state updates must be equivalent for both CLOSUREX execution and fresh process execution.

2. Control-flow equivalence: program execution path must be equivalent for both CLOSUREX execution and fresh process execution.

To determine whether CLOSUREX correctly restores state, we use the entire test case queue accumulated during the fuzzing campaign of a target. This ensures that we cover all seen program behaviors during fuzzing. For each input in the queue, we compare two state snapshots from the target. The first snapshot is taken after executing the target in a fresh process. We take the second snapshot after executing, in CLOSUREX’s persistent mode, the test case after 1000 iterations of test cases randomly selected from the queue. If CLOSUREX correctly restores state, then the two snapshots are identical.

## Dataflow equivalence

There are three sources of program state that CLOSUREX must manage in our benchmark corpus. The program stack is controlled by well-formed function calls in CLOSUREX’s harness, as well as the arguments passed to the target’s main function is correct at every execution. This ensures that the stack is fresh for every iteration.

CLOSUREX frees all the stale heap objects after every iteration of the target. To verify that the dynamic memory of the target is correctly reset by CLOSUREX, we use valgrind [21], a debugging and profiling tool, to check for dynamic memory related inconsistencies like double-free or use-after-free. We find no such occurrences for any queue input in any of our benchmarks. We also compare the memory usage of the target during its execution to a fresh process’s execution. Excluding CLOSUREX’s own memory, the memory usage of the target is identical to a fresh process execution, showcasing that the dynamically allocated memory was freed correctly.

To verify global state equivalence, we compare the global state in the first snapshot (taken after execution of a queue input in a fresh process) to the second snapshot. If the two global states match, we establish dataflow equivalence between CLOSUREX’s persistent mode execution and a correct fresh process execution. To handle non-determinism in program execution, which happens naturally even outside of fuzzing, we execute a target in a fresh process multiple times, and compare the global state for each execution. This captures ground-truth non-determinism: we consider the bytes that vary between these fresh executions non-deterministic. The reason for the existence of such non-deterministic bytes could be storing address of some data on the heap, or data generated or processed through the PRNGs.

All the targets have the same snapshot as a fresh process execution for all the fuzzing queue entries. This establishes that CLOSUREX is dataflow equivalent to a fresh process execution.

## Control-flow equivalence

We take a similar approach to dataflow equivalence to verify the execution path equivalence to a fresh process

execution. For every test case in the queue, we first get the ground-truth path sensitive edge-coverage by running the target with the queue input in a fresh process. Then we iterate over CLOSUREX persistent mode and execute 1000 other randomly-selected queue inputs. That same process is then used to execute the test case of interest and its path sensitive edge-coverage is recorded. We then compare this coverage data with the ground truth coverage for the queue input. If these two coverages are equal, CLOSUREX and fresh process execution are control-flow equivalent for this test case. We repeat this process, independently, for all the test cases in the queue. We find that no queue entry leads to an inconsistency between CLOSUREX execution and a fresh process execution.

As in dataflow equivalency, we exclude test cases that induce a non-deterministic execution path on the target, i.e., their execution path differs across multiple fresh process executions. To determine the ground-truth non-deterministic queue test cases, we run each test case 10 times within a fresh process and collect the path-sensitive edge-coverage. The test cases which generate a discrepancy among these fresh executions are then flagged as non-deterministic. We only notice such non-deterministic queue inputs in the freetype benchmark. We suspect that a PRNG affected control-flow sequence is the root cause.

We find no inconsistency in control-flow equivalence compared to fresh process execution. Given no deviation in data- or control-flow behavior, we claim that CLOSUREX ensures semantic consistency: test cases behave as if they were executed in a fresh process.

## 7 Discussion

### 7.1 Deferred Initialization

Our research shows that most of the programs have an initialization phase which is independent of user input, and ideally needs to be executed only once during fuzzing. To provide a greater speed-up for applications that exhibit this behavior, a more aggressive form of code motion is needed to isolate and pull initialization steps outside of the target program into the stub harness at a deferred initialization point inside of the persistent fuzzing loop. Implementing this is non-trivial and requires additional research into flow analysis and methodologies for hoisting large blocks of complex code outside of its current context while maintaining semantic accuracy. When successful, this enhancement allows for the elimination of test-case-independent initialization steps, improving performance.

### 7.2 Supporting Other Operating Systems

While we have implemented CLOSUREX on the Linux operating system, our approach is operating system agnostic, and requires little engineering effort to support other operating systems. Consider the Windows operating system:

CLOSUREX depends only on LLVM APIs and does not require any operating system level primitives to function. LLVM officially supports the Windows operating system and the CLOSUREX compile-time passes are directly applicable to Windows. To extend CLOSUREX's support to Windows (or to any new OS), OS-specific functions like `HeapAlloc` and `VirtualAlloc` need to be hooked in a similar way to `malloc` and `free`. Our techniques for tracking and restoring C language APIs remain unchanged across OSes. Our technique of resetting global state of a target by moving it to a new section is also directly portable to Windows and other OSes. We expect similar or better results on the Windows operating system due to its lack of a copy-on-write based fork.

### 7.3 Extending CLOSUREX

Our benchmark targets only have four sources of latent program state: local memory, global memory, dynamic memory, and file descriptors. We acknowledge that there are other sources for other programs. For example, in multi-threaded processes, if the main thread spawns multiple threads, which continue executing after the main thread exits, the operating system kills all such spawned threads. However, in persistent mode execution, the main thread never exits but loops back and such child threads continue executing until they're killed manually. To deal with such multi-threaded programs, we use a similar approach to dynamic memory resetting: we track all the threads that are created during an execution and kill any threads that remain after the main thread completes execution (this would be when they are killed by the operating system). This is just one example of how we extend the presented patterns to handle new sources of latent state.

We open-source CLOSUREX and its artifacts on GitHub [23]. We are currently creating a CLOSUREX port for U.S. government use cases and expect to see ports for different OSes, fuzzers, architectures, and even programming languages due to the low engineering effort of extending CLOSUREX.

### 7.4 Handling More Complex Program State

CLOSUREX is able to handle the most essential source of latent state in a program, our open source framework also allows to cover a lot more program states. However, there are some program states that require more target-tailored approach, some of which are:

- Source-unavailable libraries: CLOSUREX does not handle third-party code which comes pre-compiled and is later linked in with the target object file. Since we follow a compile-time approach to insert state tracking and resetting code in the program, we must have access to all source code under test.
- Storing/retrieving state from a file: The majority of fuzzing literature focuses on stateless programs, i.e., programs that start from a constant state and take

inputs in some way (e.g., command line or a file). Exploring effective fuzzing of stateful programs is an emerging area which is not in the scope of CLOSUREX.

- Custom memory allocators: An example of complex program state that CLOSUREX currently does not handle is custom memory allocators, which will require a more target-tailored approach to solve.

## 8 Related Work

CLOSUREX is orthogonal to many other fuzzing improvements, and can be combined together to achieve an even higher performance. Thus, the focus of this section is on competing process management schemes and on orthogonal fuzzing performance improvements.

### 8.1 Linux-kernel-based Process Management

Kernel-based snapshotting is an execution technique of restoring the state of the process to a prior taken "snapshot". Work done in "Designing new operating primitives" implements a new system call `snapshot`, which stores the important aspects of program state like process pages, `brk` values and restores them after every execution [34]. This technique is also implemented as a Linux Kernel Module in AFL++ [7]. Implementing this technique for closed-source kernels like Windows or MacOS is infeasible. Even though Linux is an open-source operating system with a rich API for tooling, such a snapshot process still requires a significant engineering effort to maintain it for the ever-changing Linux kernel, hence why we can only find one kernel version that works with kernel-based snapshotting. The same issue is also present in AFL-LKM [7].

### 8.2 Binary-only Windows Process Management

Recent work has introduced binary-level snapshotting for Windows programs, where a binary rewriter adds code to checkpoint and restore a binary's initial state [29]. WinFuzz is designed for tackling the lack of a fork system call on Windows without the need to modify the Windows kernel. Even though CLOSUREX improves performance over state-of-the-art forkserver approaches a comparison to WinFuzz would be interesting as CLOSUREX also support Windows programs with minimal engineering. Unfortunately, WinFuzz is not yet open-source, although qualitatively, CLOSUREX should outperform WinFuzz due to WinFuzz being a binary-only solution requiring runtime code injection overhead and imprecision with its state checkpoints, while CLOSUREX inserts all the snapshotting code at compile-time—which is further optimised during compilation—and can perform fine-grain snapshotting.

### 8.3 Less Instrumentation

Previous work shows that reducing the number of instrumented points in the program via dynamic rewriting of binaries increases the throughput of the targeted applications to effectively no overhead in the common case [11, 18–20]. CLOSUREX is orthogonal to any coverage collecting mechanism; combining CLOSUREX with a coverage-guided tracing approach will further magnify the performance benefit of CLOSUREX given that reducing target execution time creates a relative increase in process management overhead—which CLOSUREX tackles.

### 8.4 Faster Execution for Multi-core Fuzzing

Multi-core fuzzing benefits from CLOSUREX. Being able to parallelize fuzzing efforts is greatly dependent on managing OS resources [10, 34, 35]. By instrumenting the program with CLOSUREX, we have multiple self managing instances running in parallel maintaining optimal resource utilization, thus removing kernel and OS-related contentions completely. Without CLOSUREX, resource starvation impacts the performance of all fuzzing efforts leading to reduced throughput. We expect to unlock further improvements with CLOSUREX instrumentation. Similar scalability improvements like dual-filesystem for better in-target test case operations, as well as in-memory test case logging for faster test case sharing between fuzzer instances are compatible with CLOSUREX and will improve the scalability of multi-core fuzzing [34].

## 9 Conclusion

Advancements in reducing time spent tracing code coverage have improved fuzzing's performance such that process management is as a significant source of run-time overhead in fuzzing. We show that through compiler-level program instrumentation, it is possible to create naturally restartable programs that allow an entire fuzzing campaign's worth of test cases to execute within a single process. Doing so simplifies fuzzing, increases performance and bug finding, and makes high-performance fuzzing independent of operating system features. The evaluation results indicate more opportunities for fuzzer improvements through more advanced, fuzzing-specific compiler analysis.

## 10 Acknowledgment

We would like to thank the reviewers for their valuable comments and our shepherd Dr. Antony Hosking for his guidance and collaboration. This material is based upon work supported by the National Science Foundation under Grant No. 2046589.

## References

- [1] Marcel Böhme, Valentin JM Manès, and Sang Kil Cha. Boosting fuzzer efficiency: An information theoretic perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference*

- and *Symposium on the Foundations of Software Engineering*, pages 678–689, 2020.
- [2] Sang Kil Cha, Maverick Woo, and David Brumley. Program-adaptive mutational fuzzing. In *2015 IEEE Symposium on Security and Privacy*, pages 725–741, 2015.
  - [3] Chen Chen, Han Xu, and Baojiang Cui. PsOfuzzer: A target-oriented software vulnerability detection technology based on particle swarm optimization. *Applied Sciences*, 11(3), 2021.
  - [4] Sushant Dinesh, Nathan Burrow, Dongyan Xu, and Mathias Payer. Retrowrite: Statically instrumenting cots binaries for fuzzing and sanitization. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1497–1511, 2020.
  - [5] Andrea Fioraldi. Frida api fuzzer, 2020.
  - [6] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, USENIX Association, August 2020.
  - [7] Andrea Fioraldi, Marc van Hauser, Joey Jiao, and Heiko Eißfeldt. Afl++ snapshot lkm. <https://github.com/AFLplusplus/AFL-Snapshot-LKM>, 2020.
  - [8] Erol Gelenbe. A model of roll-back recovery with multiple checkpoints. In *Proceedings of the 2nd International Conference on Software Engineering, ICSE '76*, page 251–255, Washington, DC, USA, 1976. IEEE Computer Society Press.
  - [9] Google. Afl, 2017.
  - [10] Google. cluster-fuzz, 2022.
  - [11] Jinho Jung, Stephen Tong, Hong Hu, Jungwon Lim, Yonghui Jin, and Taesoo Kim. Winnie : Fuzzing windows applications with harness synthesis and fast cloning. In *NDSS*, 2021.
  - [12] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. 2018.
  - [13] C. Lattner and V. Adve. Llvm: a compilation framework for lifelong program analysis and transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, pages 75–86, 2004.
  - [14] LLVM. Libfuzzer – a library for coverage-guided fuzz testing., 2022.
  - [15] Patrick E. McKnight and Julius Najab. *Mann-Whitney U Test*, pages 1–1. John Wiley & Sons, Ltd, 2010.
  - [16] Jonathan Metzman, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. Fuzzbench: An open fuzzer benchmarking platform and service. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2021*, page 1393–1403, New York, NY, USA, 2021. Association for Computing Machinery.
  - [17] Microsoft. Onefuzz, 2022.
  - [18] Stefan Nagy and Matthew Hicks. Full-speed fuzzing: Reducing fuzzing overhead through coverage-guided tracing. *2019 IEEE Symposium on Security and Privacy (SP)*, pages 787–802, 2019.
  - [19] Stefan Nagy, Anh Nguyen-Tuong, Jason D. Hiser, Jack W. Davidson, and Matthew Hicks. Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1683–1700. USENIX Association, August 2021.
  - [20] Stefan Nagy, Anh Nguyen-Tuong, Jason D. Hiser, Jack W. Davidson, and Matthew Hicks. Same coverage, less bloat: Accelerating binary-only fuzzing with coverage-preserving coverage-guided tracing. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS '21*, page 351–365, New York, NY, USA, 2021. Association for Computing Machinery.
  - [21] Nicholas Nethercote and Julian Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '07*, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.
  - [22] Norman Peitek, Sven Apel, Chris Parnin, André Brechmann, and Janet Siegmund. *Program Comprehension and Code Complexity Metrics: A Replication Package of an FMRI Study*, page 168–169. IEEE Press, 2021.
  - [23] Matthew Hicks, Rishi Ranjan, Ian Paterson. ClosureX: Compiler Support for Correct Persistent Fuzzing. <https://github.com/FoRTE-Research/closure>.
  - [24] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. Addresssanitizer: A fast address sanity checker. In *Proceedings of the 2012 USENIX Conference on Annual Technical Conference, USENIX ATC'12*, page 28, USA, 2012. USENIX Association.
  - [25] Kosta Serebryany. Continuous fuzzing with libfuzzer and addresssanitizer. In *2016 IEEE Cybersecurity Development (SecDev)*, pages 157–157, 2016.
  - [26] Kostya Serebryany. OSS-Fuzz - google's continuous fuzzing service for open source software. Vancouver, BC, August 2017. USENIX Association.
  - [27] Prashast Srivastava and Mathias Payer. Gramatron: Effective grammar-aware fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2021*, page 244–256, New York, NY, USA, 2021. Association for Computing Machinery.
  - [28] Richard M. Stallman and GCC DeveloperCommunity. *Using The Gnu Compiler Collection: A Gnu Manual For Gcc Version 4.3.3*. CreateSpace, Scotts Valley, CA, 2009.
  - [29] Leo Stone, Rishi Ranjan, Stefan Nagy, and Matthew Hicks. No linux, no problem: Fast and correct windows binary fuzzing via target-embedded snapshotting. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4913–4929, Anaheim, CA, August 2023. USENIX Association.
  - [30] The Clang Team. Sanitizercoverage, 2019.
  - [31] Peach Tech. “peach fuzzing platform”, 2021.
  - [32] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Skyfire: Data-driven seed generation for fuzzing. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 579–594, 2017.
  - [33] Matthias Wenzl, Georg Merzdovnik, Johanna Ullrich, and Edgar Weippl. From hack to elaborate technique—a survey on binary rewriting. *ACM Comput. Surv.*, 52(3), jun 2019.
  - [34] Wen Xu, Sanidhya Kashyap, Changwoo Min, and Taesoo Kim. Designing new operating primitives to improve fuzzing performance. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, page 2313–2328, New York, NY, USA, 2017. Association for Computing Machinery.
  - [35] M. Zalewski. Technical whitepaper on afl-fuzz. 2017.
  - [36] Zhuo Zhang, Wei You, Guanhong Tao, Yousra Aafer, Xuwei Liu, and Xiangyu Zhang. Stochfuzz: Sound and cost-effective fuzzing of stripped binaries by incremental and stochastic rewriting. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 659–676, 2021.